Unit 1

Introduction to Software Engineering

Prerequisite

Software Engineering Definition

Software is a collection of programs, procedures, and related data that tell a computer what to do and how to perform tasks.

Systematic, disciplined ,cost effective technique for s/w development.

Engineering approach to develop s/w

Ex. Branded and non branded

Characteristics

1. **Intangible:** Software cannot be physically touched or seen like hardware.
2. **Custom-built:** Software is designed and developed based on user requirements, unlike mass-produced hardware.
3. **Evolves Over Time:** Software is constantly updated and improved to meet new requirements and fix issues.
4. **No Wear & Tear:** Unlike hardware, software does not degrade but may become obsolete due to new technology.
5. **Quality Depends on Development:** Well-structured software is reliable, efficient, and maintainable.

Application Domains

- **System Software:** Manages hardware resources and provides a platform for application software.Application Software: Solves specific business needs and processes business or technical data.
- **Engineering/Scientific Software:** Facilitates engineering and scientific tasks, moving away from conventional numerical algorithms.
- **Embedded Software:** Implements and controls features and functions within a system or product.
- **Web Applications:** Provides a wide array of applications that are network-centric and evolve continuously.

Why Study Software Engineering?

Ability to solve complex programming problems:

How to break large projects into smaller and manageable parts?

Learn techniques of:

specification, design, interface development, testing, project management, etc.

To acquire skills to be a better programmer:

- Higher Productivity
- Better Quality Programs

Evolution of Software Engineering

- 1945-1965: **Origin**
- 1965-1985: **Crisis**
- 1990-2000: **Internet**
- 2000-2010: **Lightweight**
- 2010- till date: **AI,ML,DL**

Characteristics of Good Software - Software Engineering

What is a Good Software?

Software engineering is the process of designing, developing, and maintaining software systems. Good software meets the needs of its users, performs its intended functions reliably, and is easy to maintain.

The factors that decide the software properties are divided into three categories: **Operational, Transitional, and Maintenance.**

- **Operational**
  - Budget
  - Usability
  - Efficiency
  - Correctness
  - Functionality
  - Dependability
  - Security
  - Safety
- **Transitional**

  When the software is moved from one platform to another, the factors deciding the software quality:

  - Portability
  - Interoperability
  - Reusability
  - Adaptability
- **Maintenance**

  In this categories all factors are included that describes about how well a software has the capabilities to maintain itself in the ever changing environment:

  - Modularity
  - Maintainability
  - Flexibility
  - Scalability

Characteristics of Good Software

1. **Functionality:** The software meets the requirements and specifications that it was designed for, and it behaves as expected when it is used in its intended environment.
2. **Usability:** The software is easy to use and understand, and it provides a positive user experience.
3. **Reliability:** The software is free of defects and it performs consistently and accurately under different conditions and scenarios.
4. **Performance:** The software runs efficiently and quickly, and it can handle large amounts of data or traffic.
5. **Security:** The software is protected against unauthorized access and it keeps the data and functions safe from malicious attacks.
6. **Maintainability:** The software is easy to change and update, and it is well-documented, so that it can be understood and modified by other developers.
7. **Reusability:** The software can be reused in other projects or applications, and it is designed in a way that promotes code reuse.
8. **Scalability:** The software can handle an increasing workload and it can be easily extended to meet the changing requirements.
9. **Testability:** The software is designed in a way that makes it easy to test and validate, and it has a comprehensive test coverage.

Software Engineering Ethics and Professional Conducts

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

1. PUBLIC - Software engineers shall act consistently with the public interest.

2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.

3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.

4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.

5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.

6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.

7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.

8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Software Process Models

<u>Key Activities:</u>

Specification/ Requirement Gathering: Defining what the system should do (requirements).

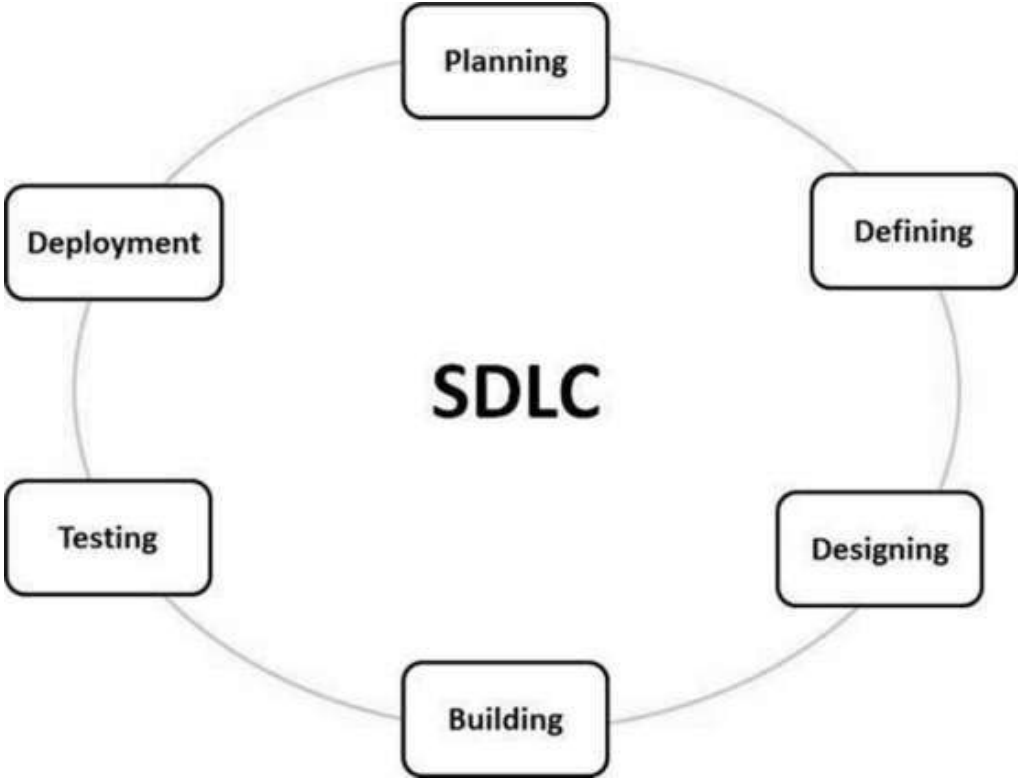
Design: Organizing the system's structure.

Implementation/ Development: Writing the code.

Validation (Testing): Checking if it meets customer needs.

Evolution: Modifying the system as needs change.

SDLC


Planning
Defining
Designing
Building
Testing
Deployment
SDLC

Common Software Process Models

Waterfall Model: A sequential, linear approach where each phase (requirements, design, coding, testing) must finish before the next begins.

V-Model: An extension of Waterfall, emphasizing testing by associating each development phase with a corresponding testing phase (e.g., design with integration testing).

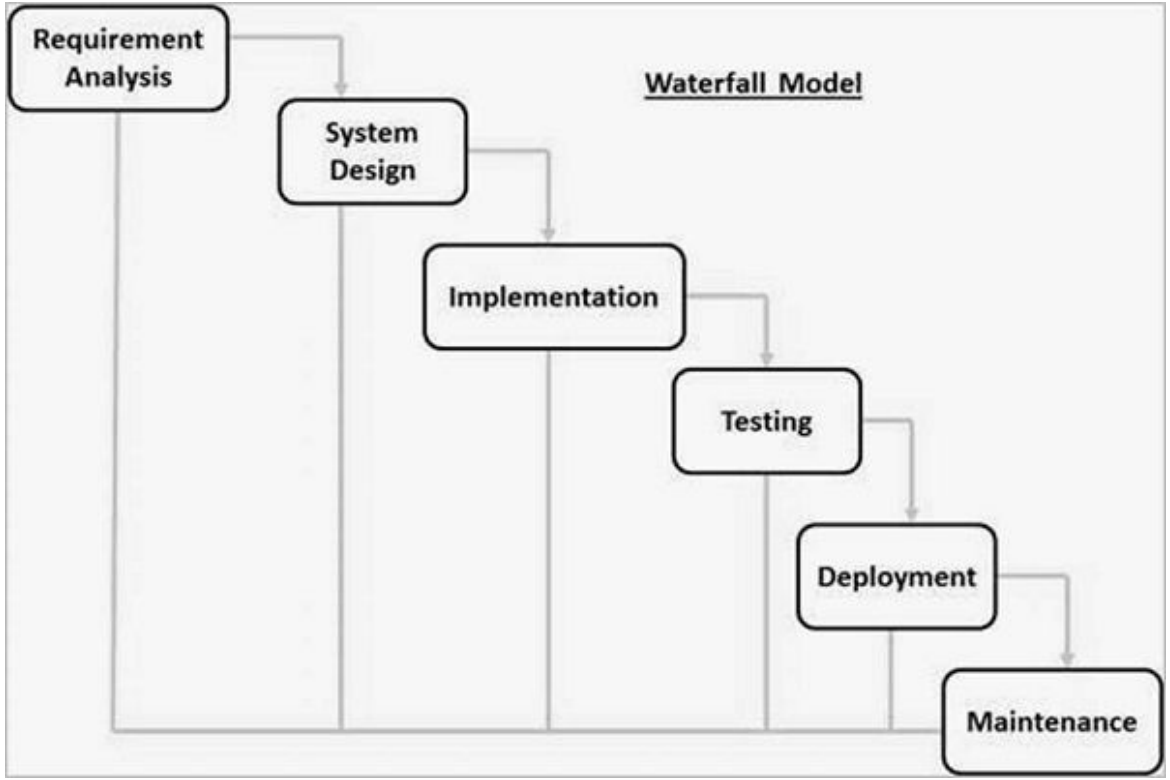
Incremental Model: Delivers the system in small, usable increments, with each addition building on the last.

Iterative Model: Repeats cycles of development (design, implement, test) to refine the system, often using prototyping.

Spiral Model: Combines iterative development with systematic risk management, circling through planning, risk analysis, engineering, and evaluation.

Agile/RAD (Rapid Application Development): Focuses on quick iterations, user feedback, and adaptability to deliver working software faster.

Waterfall Model - Software Engineering



Phases of Waterfall Model

1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer’s needs.

- **Requirement Gathering and Analysis:**

Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.

- **Requirement Specification:**

The approved requirements are documented in the Software Requirement Specification (SRS), which serves as a formal agreement between the customer and the development team.

2. Design

In this phase, the requirements from the SRS are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**

  Defines the overall system architecture, major components, and their interactions.

- **Low-Level Design (LLD):**

  Provides detailed design of each component, including logic and data flow, to guide developers during coding.

  All designs are documented in the Software Design Document (SDD).

3. Development

This phase involves converting the design into actual working software.Developers write source code based on the design documents. Suitable programming languages and tools are used.

Unit testing is performed to verify that each module works correctly on its own.

4. Testing and Deployment

This phase ensures that the integrated software functions correctly and is successfully delivered for real world use.

Testing: After unit testing, software modules are integrated incrementally and tested at each stage. Once all modules are successfully integrated, full system testing is performed to ensure the system functions as expected.

Alpha Testing: Performed by the development team.

Beta Testing: Performed by selected end users.

Acceptance Testing: Performed by the customer to approve the software.

Deployment: After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

Features of the Waterfall Model

Sequential Approach

Development follows a linear, step-by-step process where each phase is completed before moving to the next.

Document-Driven Process

Detailed documentation is created at every stage to clearly define requirements, design, and progress.

Quality Control

Strong emphasis is placed on verification and testing to ensure each phase meets its defined requirements.

Detailed Planning

The project scope, schedule, and deliverables are carefully planned and monitored throughout the lifecycle.

Advantages

- **Simple and easy to understand and use.**
- Easy to manage due to the rigidity of the model.
- Phases are processed and completed one at a time.
- **Works well for smaller projects where requirements are very well understood.**
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

Disadvantages

- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.
- Integration is done as a "big-bang. at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.

Incremental Model

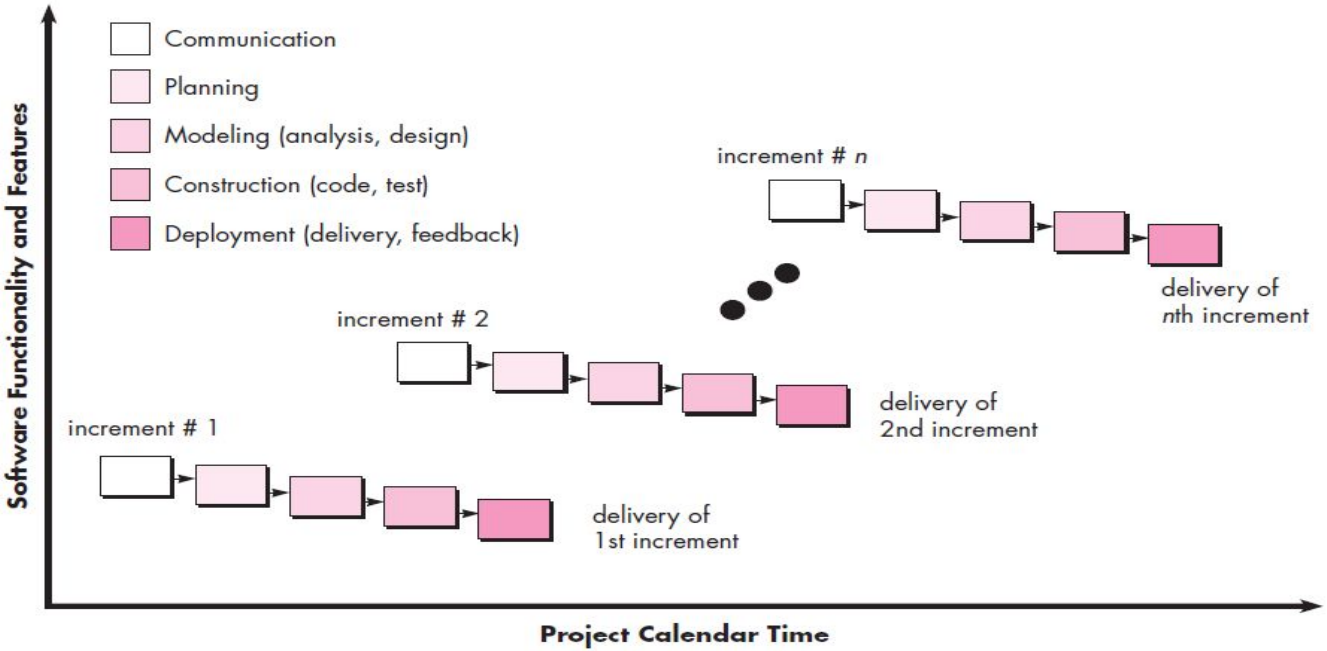
The Incremental Process Model breaks down a system into smaller portions, known as increments.

Each increment adds functional capabilities to the product.

The idea is to produce a working version of the software early, then improve and expand it with successive increments.

- **Module by Module Development**
- **Maximum customer interaction**
- **Beneficial for Early release product demand**
- **Flexible to changes**

Incremental Model



Advantages

1. Works in small size team.
2. Initial product delivery is fast.
3. Customer feedback considered.
4. Accommodate changes easily.

Disadvantages

1. Actual cost may exceed the estimated cost
2. System broken into small framework. It needs strong designing.

V-model

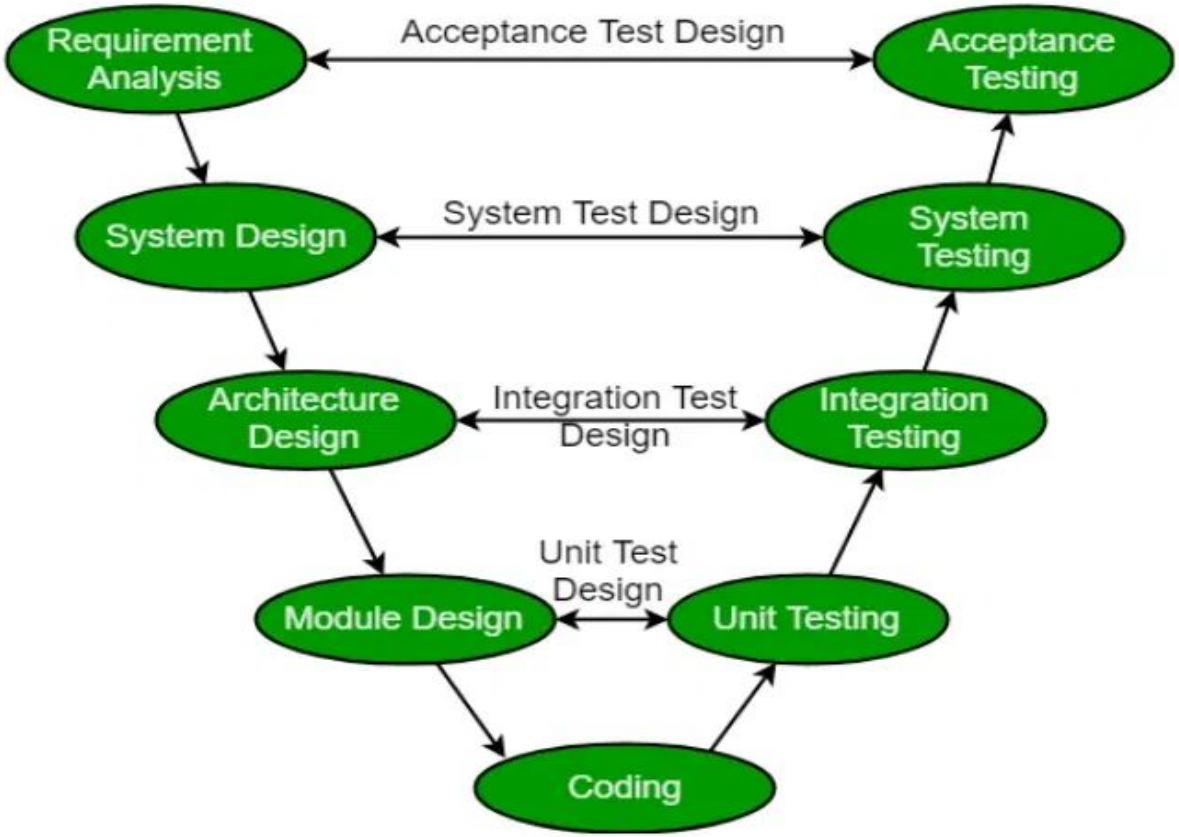
The V-Model, which includes the Verification and Validation it is a structural approach to software development.

Extension of waterfall model.

Testing is associated with each phase of the model.

Verification Phase(Requirement Analysis, System Design, Architecture Design, Module Design).

Validation Phase (Unit Testing, Integration Testing, System Testing, Acceptance Testing)


Requirement Analysis
Acceptance Test Design
Acceptance Testing
System Design
System Test Design
System Testing
Architecture Design
Integration Test Design
Integration Testing
Unit Test Design
Module Design
Unit Testing
Coding

V- Model Application

- Requirements are well defined, clearly documented and fixed.
- Product definition is stable.
- Technology is not dynamic and is well understood by the project team.
- There are no ambiguous or undefined requirements.
- The project is short.

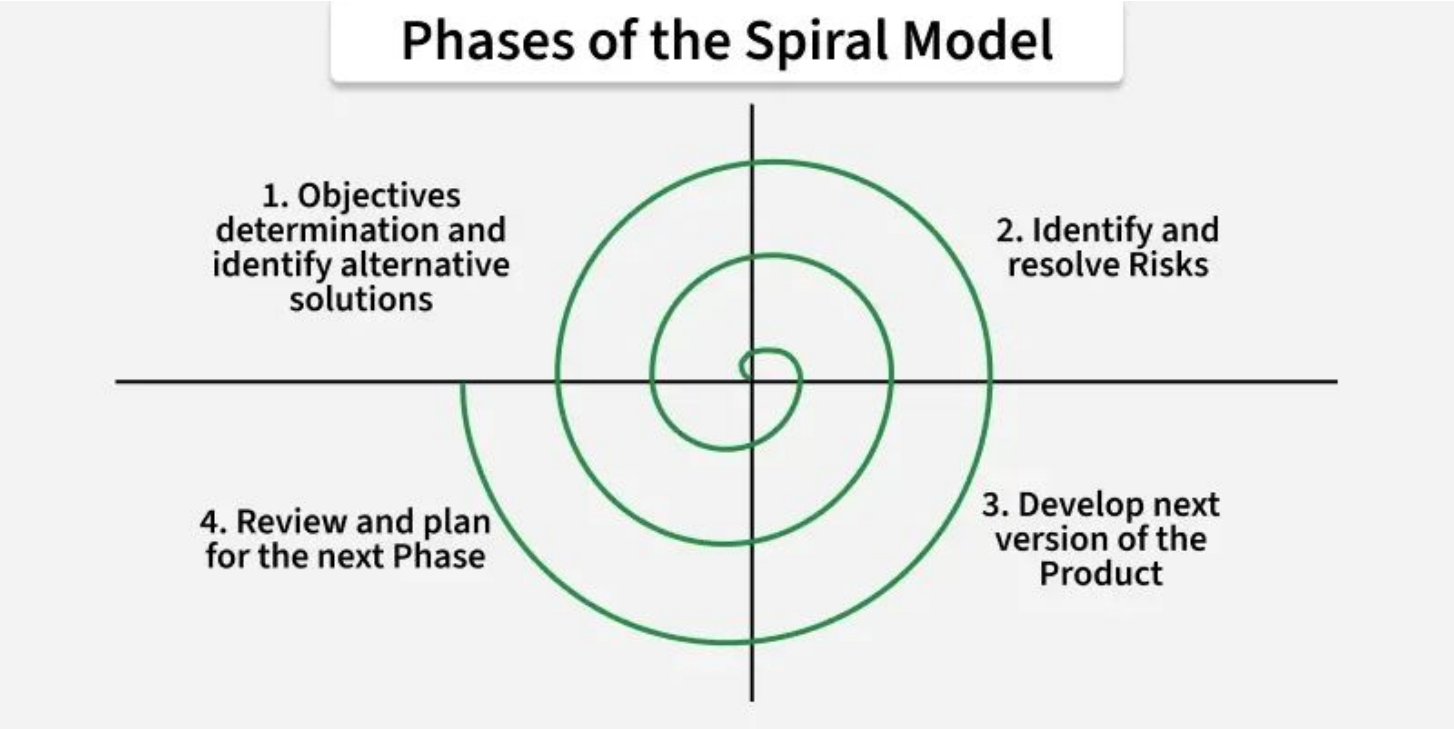
Advantages

- This is a highly-disciplined model and Phases are completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Simple and easy to understand and use.
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.

Disadvantages

- High risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- No customer interaction during the process.
- No working software is produced until late during the life cycle.

Spiral Model



Spiral Model

The Spiral Model is a risk-driven and iterative software development model. Development progresses in a spiral shape consisting of multiple loops, where each loop represents a complete development cycle.

The number of loops depends on project size, complexity, and risk.

Each loop includes:

- Planning
- Risk analysis
- Development
- Evaluation

*The Spiral model is called a **Meta-Model** because it subsumes all the other SDLC models.*

Phases of the Spiral Model

Focus is on managing risk through multiple iterations of the software development process. Each phase of the **Spiral Model is divided into four Quadrants.**

1. Objectives Definition

- Project goals and requirements are identified
- Functional and non-functional requirements are analyzed
- Possible solutions are explored

2. Risk Analysis and Resolution

- Risks related to cost, schedule, performance, and technology are identified
- Best solution is selected
- Prototypes are built to reduce risks

3. Development and Testing

- Selected features are designed, developed, and tested
- The next working version of the software is created

4. Review and Planning

- Customers evaluate the current version
- Feedback is collected
- Planning for the next iteration begins

The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.

Advantages

1. **Effective Risk Handling:** Identifies and resolves risks at every phase, making it ideal for projects with high uncertainty.
2. **Suitable for Large Projects:** Best suited for large and complex software systems.
3. **Flexible Requirements:** Allows changes in requirements even at later stages of development.
4. **Early Customer Feedback:** Customers can review early versions of the product, improving satisfaction.
5. **Iterative Development:** Software is developed in repeated cycles, allowing gradual improvement.
6. **Better Communication:** Regular reviews improve coordination between customers and developers.
7. **High Software Quality:** Continuous testing and refinement lead to reliable and high-quality software.

Disadvantages

1. **Complex to Manage:** The model is complex due to multiple iterations and continuous risk analysis.
2. **High Cost:** Not suitable for small projects because it requires more time, effort, and resources.
3. **Depends Heavily on Risk Analysis:** Success depends on accurate risk identification, which requires experienced professionals.
4. **Difficult Time Estimation:** The number of iterations is not fixed, making scheduling difficult.
5. **Time-Consuming:** Frequent reviews and evaluations can increase development time.
6. **Resource Intensive:** Requires significant investment in planning, risk management, and testing.

Example of the Spiral Model

Smart Agriculture Monitoring System

Consider the example of developing a system that will help farmers in monitoring soil moisture, temperature, and weather data using IoT sensors and a mobile app.

First Spiral Cycle

The team identified sufficient basic needs, that is, they have to monitor soil moisture and view results in real time in a mobile app; Some risks were identified, such as poor internet connectivity, exposure of sensors to weather conditions, and delayed data transmission. A super basic prototype was built with one moisture sensor and a simple app interface. The prototype was tested in a controlled environment for connectivity and real-time updates.

Second Spiral Cycle

New functionality was added, including temperature sensor functionality and new alerts for low soil moisture to plan for irrigation better. This version was tested in a medium-sized farm to see how it performed in the field and how it consumed power.

Third Spiral Cycle

The system was expanded to include multiple farm zones and local weather data for planning purposes. Multilingual features were added to help more users overcome language barriers. User testing revealed problems with sensor pairing and a complicated onboarding process. To fix this, the onboarding was simplified, and the app interface was redesigned with accessible elements to improve the overall experience.

Fourth Spiral Cycle

The focus of the team shifted to battery life improvement, saving data in offline mode, and app speed improvement. Usage analytics are introduced to refine user-oriented features based on real user behavior. They also addressed data privacy protections are put in place and began to include supporting videos and guides for farmers who are new to technology.

Agile Model

The Agile Model was primarily designed to help a project adapt quickly to change requests. So, the main aim of the Agile model is to facilitate quick project completion. To accomplish this task, it's important that agility is required. Agility is achieved by fitting the process to the project and removing activities that may not be essential for a specific project.

Steps in the Agile Model

The Agile Model is a combination of iterative and incremental process models. The phases involved in Agile (SDLC) Model are:

- **Requirement Gathering**
- **Design the Requirements**
- **Construction / Iteration**
- **Testing / Quality Assurance**
- **Deployment**
- **Feedback**

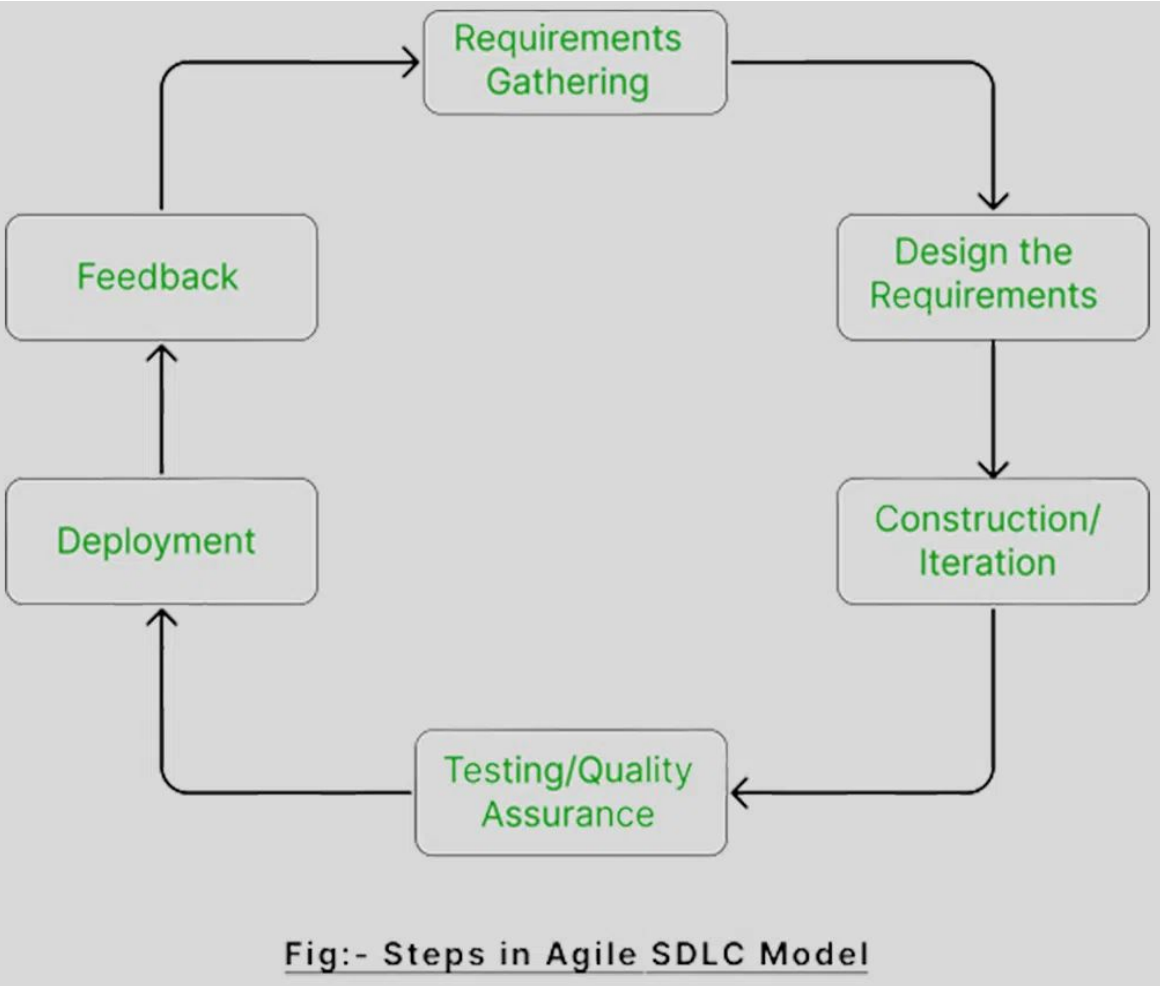
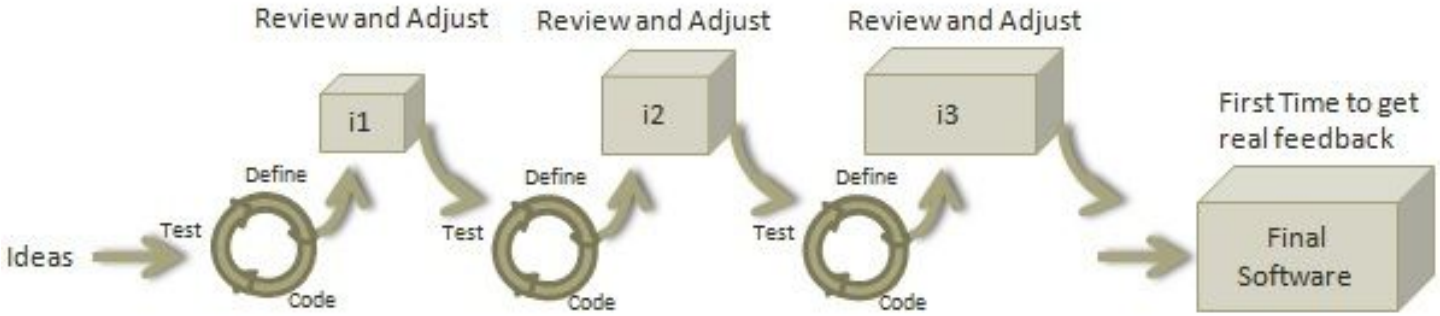


Fig:- Steps in Agile SDLC Model

Traditional Method



Agile Method

Advantages

- Working through Pair programming produces well-written compact programs which have fewer errors as compared to programmers working alone.
- Less time consumption
- More customer interaction
- Adaptive for changes
- Customer centric approach , it helps to meet their needs.

Disadvantages

- Lack of documentation
- It works in short sprint, which can make forecasting of project difficult.
- More reliable on customer interaction, if customer is not having clear idea then the development team can be driven wrong decision.